Unity Programmer
Pre-Work Reference Guide

Architecture · Performance · Systems · Tooling

A. Architecture

S-O-L-I-D Principles

Why?	SOLID splits your code into small, single-responsibility, independent pieces. You can extend without breaking existing code when adding new features. Merge conflicts decrease in team settings and writing tests becomes easier.
Without it?	Classes bloat; a change in one place breaks others. Over time 'spaghetti code' forms — changing a weapon requires touching 5 different classes.
Where?	Enemy, Player, Weapon, Ability System, UIManager class designs. Before writing any new system, ask: 'What is the single responsibility of this class?'

In-Game Usage Example

```
// S - Single Responsibility: Only calculates damage public class DamageCalculator { public float Calculate(float baseDmg, float multiplier) => baseDmg * multiplier; } // O - Open/Closed: Add weapon types without modifying existing code public abstract class WeaponBase { public abstract void Fire(); } public class Bow : WeaponBase { public override void Fire() { /* shoots arrow */ } } public class Staff : WeaponBase { public override void Fire() { /* casts spell */ } } // L - Liskov: Subtypes must be substitutable for base types // I - Interface: IMovable, IDamageable kept separate // D - Dependency Inversion: Depend on abstractions, not concrete classes
```

Abstraction

Why?	Prevents consumer code from knowing implementation details. When a service changes (e.g. a different audio engine), code that uses the interface stays unchanged. Makes mocking easy.
Without it?	Every system becomes tightly coupled. Swapping the audio system means updating 50 call sites.
Where?	Use for interface definitions such as IAudioService, ISaveService, IInputProvider, IEnemyAI.

In-Game Usage Example

```
// Interface (abstraction layer) public interface IAudioService { void Play(string clipName); void Stop(string clipName); } // Concrete implementation public class UnityAudioService : IAudioService { public void Play(string clipName) { /* AudioSource.Play... */ } public void Stop(string clipName) { /* ... */ } } // Consumer only knows the interface public class Player { private IAudioService _audio; public Player(IAudioService audio) => _audio = audio; void OnJump() => _audio.Play("jump_sfx"); }
```

Singleton Pattern

Why?	Provides a global access point for systems that need exactly one instance across the game (GameManager, AudioManager, EventBus). Survives scene transitions when used correctly.
Without it?	You end up calling Find() or GetComponent() everywhere to reach the same service — hurting performance and increasing null reference errors.
Where?	GameManager, AudioManager, SceneLoader, EventBus. Don't make everything a singleton; only truly global systems.

In-Game Usage Example

```
public class GameManager : MonoBehaviour { public static GameManager Instance { get; private set; } void Awake() { if (Instance != null) { Destroy(gameObject); return; } Instance = this; DontDestroyOnLoad(gameObject); } public void PauseGame() { Time.timeScale = 0f; } public void ResumeGame() { Time.timeScale = 1f; } } // Usage: GameManager.Instance.PauseGame();
```

ScriptableObject Architecture

Why?	Stores game data (enemy stats, item configs, level data) outside code as Inspector-editable assets. Designers/balancers can work without touching code. Preserves original data at runtime without cloning.
Without it?	Values buried in prefabs get messy; you have to open the prefab to change a value. You end up storing the same data in multiple places.
Where?	EnemyData, WeaponConfig, LevelConfig, AbilityData, ItemDatabase — all game data assets.

In-Game Usage Example

```
[CreateAssetMenu(menuName = "Game/EnemyData")] public class EnemyData : ScriptableObject { public string enemyName; public float maxHealth; public float moveSpeed; public float attackDamage; public float spawnWeight; } // Enemy reads its data from the SO public class Enemy : MonoBehaviour { [SerializeField] private EnemyData _data; private float _currentHealth; void Start() => _currentHealth = _data.maxHealth; public void TakeDamage(float dmg) => _currentHealth -= dmg; }
```

Factory Pattern

Why?	Centralises object-creation logic. You can decide which type to create at runtime. Integrates cleanly with object pooling.
Without it?	Instantiate calls scatter everywhere; you repeat the same spawn logic in different places (DRY violation) and changes become hard.
Where?	EnemyFactory, ProjectileFactory, VFXFactory, UIElementFactory — centralise all Instantiate logic here.

In-Game Usage Example

```
public class EnemyFactory : MonoBehaviour { [SerializeField] private EnemyData[] _enemyTypes; [SerializeField] private Transform _spawnParent; public Enemy Create(string id, Vector3 pos) { var data = Array.Find(_enemyTypes, e => e.name == id); var go = Instantiate(data.prefab, pos, Quaternion.identity, _spawnParent); var enemy = go.GetComponent(); enemy.Initialize(data); return enemy; } }
```

Polymorphism

Why?	Different enemy types, weapons, or abilities can implement the same interface. Adding a new type doesn't change existing code — you just write a new class.
Without it?	Switch/if-else chains grow out of control. Adding a new enemy type means updating 10 different places.
Where?	Enemy AI types, Ability system, Weapon types, UI elements, effect system.

In-Game Usage Example

```
public abstract class AbilityBase : ScriptableObject { public abstract void Activate(PlayerController player); public abstract void Deactivate(PlayerController player); } public class DashAbility : AbilityBase { public float dashForce = 20f; public override void Activate(PlayerController p) { /* apply dash */ } public override void Deactivate(PlayerController p) { } } public class ShieldAbility : AbilityBase { public override void Activate(PlayerController p) { /* open shield */ } public override void Deactivate(PlayerController p) { /* close shield */ } } // Player uses all abilities the same way: // foreach (var ability in equippedAbilities) ability.Activate(player);
```

Base Entity Definitions

Why?	Defines common behaviour (health, damage, death) in one place; all entities inherit from it. No code duplication — one change affects all entities.
Without it?	You write a separate health system for every class. Fixing a bug means updating 10 different classes.
Where?	Entity (ancestor of all game objects), DamageableEntity, MovableEntity, InteractableEntity.

In-Game Usage Example

```
public abstract class Entity : MonoBehaviour { protected EntityData data; protected float currentHealth; public bool IsAlive => currentHealth > 0f; public virtual void Initialize(EntityData d) { data = d; currentHealth = d.maxHealth; } public virtual void TakeDamage(float amount) { currentHealth = Mathf.Max(0, currentHealth - amount); if (!IsAlive) OnDeath(); } protected abstract void OnDeath(); } public class Goblin : Entity { protected override void OnDeath() { DropLoot(); VFXFactory.Spawn("death_vfx", transform.position); gameObject.SetActive(false); } }
```

Decoupling

Why?	Removes direct dependencies between systems. With EventBus or a messaging pattern, components communicate without knowing each other. Testing, swapping, and extending become easy.
Without it?	Player holds a direct reference to Enemy. When Enemy is destroyed you get a NullReferenceException. You can't test modules independently.
Where?	Event systems (UnityEvents, C# events, ScriptableObject events), Dependency Injection, Service Locator pattern.

In-Game Usage Example

```
[CreateAssetMenu(menuName="Events/GameEvent")] public class GameEvent : ScriptableObject { private List _listeners = new(); public void Raise() { for (int i = _listeners.Count - 1; i >= 0; i--) _listeners[i].OnEventRaised(); } public void Register(GameEventListener l) => _listeners.Add(l); public
```

```
void Unregister(GameEventListener l) => _listeners.Remove(l); } // Enemy raises event on death; knows nothing about other systems public class Enemy : Entity { [SerializeField] private GameEvent _onEnemyDied; protected override void OnDeath() => _onEnemyDied.Raise(); }
```

B. Object Pooling

Projectile Pooling

Why?	Instantiate/Destroy on every shot causes GC spikes and frame drops. With a pool, projectiles are kept ready — only activated/deactivated. Hundreds of bullets at 60 fps work without a hitch.
Without it?	Dozens of Instantiate/Destroy per second: memory fragmentation, GC pauses, and visible stutters. A critical performance issue on mobile.
Where?	Arrows, bullets, spell projectiles, laser segments — any frequently spawned/destroyed object.

In-Game Usage Example

```
public class ProjectilePool : MonoBehaviour { [SerializeField] private Projectile _prefab; [SerializeField] private int _initialSize = 30; private Queue _pool = new(); void Start() { for (int i = 0; i < _initialSize; i++) { var p = Instantiate(_prefab); p.gameObject.SetActive(false); p.Pool = this; _pool.Enqueue(p); } } public Projectile Get(Vector3 pos, Quaternion rot) { var p = _pool.Count > 0 ? _pool.Dequeue() : Instantiate(_prefab); p.transform.SetPositionAndRotation(pos, rot); p.gameObject.SetActive(true); return p; } public void Return(Projectile p) { p.gameObject.SetActive(false); _pool.Enqueue(p); } }
```

Floating Texts (Damage / Score Numbers)

Why?	Floating texts like '+150 damage' or '+50 gold' spawn frequently. A pool lets you reuse each text object. The Instantiate cost for Canvas elements is especially high.
Without it?	A UI element is Instantiated on every hit, triggering a Canvas rebuild. Significant frame drops during intense combat.
Where?	Damage numbers, critical hit indicators, gold/XP gains, debuff indicators.

In-Game Usage Example

```
public class FloatingTextPool : MonoBehaviour { [SerializeField] private FloatingText _prefab; [SerializeField] private Canvas _worldCanvas; private Stack _pool = new(); public void Show(string text, Vector3 worldPos, Color col) { var ft = _pool.Count > 0 ? _pool.Pop() : Instantiate(_prefab, _worldCanvas.transform); ft.gameObject.SetActive(true); ft.Play(text, worldPos, col, () => Return(ft)); } void Return(FloatingText ft) { ft.gameObject.SetActive(false); _pool.Push(ft); } } // FloatingTextPool.Instance.Show("-150", enemy.transform.position, Color.red);
```

VFX & Enemy Pooling

Why?	Particle effects and enemy prefabs are heavy objects. Pooling drastically reduces both CPU and memory usage. Unity's built-in pool (UnityEngine.Pool) is production-ready.
Without it?	Destroy on every enemy death, Instantiate on every VFX. Performance collapses in scenes with 50+ enemies.

Where?	ParticleSystem effects (hit, death, explosion), all enemy types, environment objects (shell casings, etc.).

In-Game Usage Example

```
using UnityEngine.Pool; public class VFXPool : MonoBehaviour { [SerializeField] private ParticleSystem _hitVFX; private IObjectPool _pool; void Awake() { _pool = new ObjectPool( createFunc: () => Instantiate(_hitVFX), actionOnGet: ps => ps.gameObject.SetActive(true), actionOnRelease: ps => ps.gameObject.SetActive(false), actionOnDestroy: ps => Destroy(ps.gameObject), defaultCapacity: 20 ); } public void PlayAt(Vector3 pos) { var ps = _pool.Get(); ps.transform.position = pos; ps.Play(); StartCoroutine(ReturnAfter(ps, ps.main.duration)); } IEnumerator ReturnAfter(ParticleSystem ps, float delay) { yield return new WaitForSeconds(delay); _pool.Release(ps); } }
```

Sound Pooling

Why?	Playing game audio with a new AudioSource each time is costly. Channel management becomes mandatory when multiple identical sounds play simultaneously.
Without it?	With a single AudioSource, starting a new sound cuts the previous one. Creating many AudioSources leads to memory issues.
Where?	SFX system (hit, shoot, pickup, footstep, UI click), ambient sound management.

In-Game Usage Example

```
public class AudioPool : MonoBehaviour { [SerializeField] private int _poolSize = 16; private Queue _sources = new(); void Awake() { for (int i = 0; i < _poolSize; i++) { var src = gameObject.AddComponent(); src.playOnAwake = false; _sources.Enqueue(src); } } public void Play(AudioClip clip, float volume = 1f) { var src = _sources.Dequeue(); src.clip = clip; src.volume = volume; src.Play(); _sources.Enqueue(src); // Circular queue } }
```

Spawn Rate & Data-Driven Enemy Control

Why?	Storing enemy spawn timing and density in a ScriptableObject lets designers tune game balance without writing code. The wave system can be curve-driven.
Without it?	Spawn values are baked into code. Rebalancing requires a programmer and every change demands a recompile.
Where?	WaveManager, EnemySpawner, DifficultyScaler — controlled via a SpawnData asset.

In-Game Usage Example

```
[CreateAssetMenu(menuName="Game/WaveData")] public class WaveData : ScriptableObject { public EnemySpawnEntry[] entries; public float waveDuration; public AnimationCurve spawnRateCurve; // spawn rate over time } [System.Serializable] public class EnemySpawnEntry { public EnemyData enemyData; public float spawnWeight; public float startAppearTime; // seconds into the wave when this enemy appears } // Spawner computes interval from the curve: // float interval = 1f / _wave.spawnRateCurve.Evaluate(elapsed / _wave.waveDuration);
```

MVC / MVP Architecture (UI & Logic Separation)

Why?	Separating Model (data), View (visuals) and Presenter/Controller (logic) means UI changes don't touch game logic and vice versa. Unit testing becomes straightforward.

Without it?	UI classes start managing game data directly. Damage calculations end up inside the health bar. Spaghetti UI code is untestable and unmaintainable.
Where?	HealthBar Presenter, Inventory Presenter, ShopPresenter, HUDPresenter — all player/enemy interfaces.

In-Game Usage Example

```
// Model: pure data public class PlayerModel { public float CurrentHealth { get; private set; } public float MaxHealth { get; private set; } public event Action OnHealthChanged; public void TakeDamage(float dmg) { CurrentHealth = Mathf.Max(0, CurrentHealth - dmg); OnHealthChanged?.Invoke(CurrentHealth, MaxHealth); } } // Presenter: listens to Model, updates View public class HealthBarPresenter : MonoBehaviour { [SerializeField] private Slider _slider; private PlayerModel _model; public void Bind(PlayerModel model) { _model = model; _model.OnHealthChanged += UpdateView; } void UpdateView(float current, float max) => _slider.value = current / max; }
```

C. Addressables

Memory Unload

Why?	Unloading unused assets from memory is essential, especially on mobile. With Addressables Release, every loaded asset can be freed automatically when its reference count hits zero.
Without it?	Boss-fight textures linger in memory after the encounter ends. Over long play sessions RAM fills up and iOS/Android kills the app.
Where?	Asset release fires on level transitions, after boss fights, and when the shop/menu closes.

In-Game Usage Example

```
private AsyncOperationHandle _handle; async void LoadBossTexture() { _handle = Addressables.LoadAssetAsync("boss_dragon"); await _handle.Task; _bossImage.sprite = _handle.Result; } // Free memory when done void UnloadBossTexture() { if (_handle.IsValid()) Addressables.Release(_handle); } // Triggered during scene unload void OnDestroy() => UnloadBossTexture();
```

Lazy Loading

Why?	The game doesn't need to load everything at startup. Loading lazily — only when needed — reduces both initial load time and instantaneous memory usage.
Without it?	Loading all assets up front = long loading screens, high RAM, and crashes on low-end devices.
Where?	Shop item visuals, undiscovered map area assets, DLC content, cutscene assets.

In-Game Usage Example

```
public class LazyItemIcon : MonoBehaviour { [SerializeField] private AssetReferenceSprite _iconRef; [SerializeField] private Image _image; private AsyncOperationHandle _handle; // Triggered only when the item becomes visible public async void Show() { _handle = _iconRef.LoadAssetAsync(); await _handle.Task; if (_handle.Status == AsyncOperationStatus.Succeeded) _image.sprite = _handle.Result; } void OnDisable() { if (_handle.IsValid()) Addressables.Release(_handle); } }
```

Sprite Bundles (Atlas Organisation)

Why?	Grouping related sprites (UI icons, enemy atlas, environment tiles) in the same bundle improves both load performance and draw call count. A sprite atlas reduces GPU batch count.
Without it?	Individual textures per icon generate hundreds of draw calls. Without bundles you lose granular asset control and unnecessary textures linger in memory.
Where?	UI icon sets, all visual assets for a level, character skin bundles.

In-Game Usage Example

```
// Addressables Groups Window: // Weapons Bundle -> sword.png, bow.png, staff.png // UI Icons Bundle -> health_icon.png, mana_icon.png // FireLand Bundle -> tileset, bg_parallax, enemy_atlas // Load all sprites in a bundle at once var handle = Addressables.LoadAssetsAsync( "UI_Icons", sprite => _iconDict[sprite.name] = sprite ); await handle.Task; // Pre-download FireLand bundle while loading the level var preload = Addressables.DownloadDependenciesAsync("FireLand"); await preload.Task; Addressables.Release(preload);
```

D. Sheets Sync

Sheets API & CSV Export

Why?	Enemy stats, item values, and dialogue text can be managed by designers/writers in Sheets. Programmers no longer need to open code for every change.
Without it?	All balancing data is buried in code or complex Inspector fields. Non-technical team members can't enter data.
Where?	EnemyStats, ItemDatabase, DialogLines, LevelConfigs, LocalizationStrings.

In-Game Usage Example

```
public static class SheetsSyncer { const string SHEET_URL = "https://docs.google.com/spreadsheets/d/YOUR_ID/export?format=csv&gid;=0"; [MenuItem("Tools/Sync Enemy Data")] static async void SyncEnemyData() { using var client = new System.Net.Http.HttpClient(); string csv = await client.GetStringAsync(SHEET_URL); ParseAndApply(csv); AssetDatabase.SaveAssets(); Debug.Log("[Sync] Enemy data updated!"); } static void ParseAndApply(string csv) { var rows = csv.Split('\n'); // rows[0] == header, rows[1+] == data foreach (var row in rows[1..]) { /* update SOs */ } } }
```

Auto Refresh (Editor Only)

Why?	Auto-pulling the sheet at set intervals or on file save while the Editor is open lets designers test their changes immediately in-game.
Without it?	Every data change requires a manual click of the sync menu; human error can lead to testing with stale data.
Where?	Editor script only (EditorApplication.update or FileSystemWatcher). Don't include in builds!

In-Game Usage Example

```
[InitializeOnLoad] public static class AutoSheetsRefresh { static double _lastCheck; const double INTERVAL = 30.0; // seconds static AutoSheetsRefresh() { EditorApplication.update += OnEditorUpdate; } static void OnEditorUpdate() { if (EditorApplication.timeSinceStartup - _lastCheck < INTERVAL) return; _lastCheck =
```

```
EditorApplication.timeSinceStartup; if (EditorApplication.isPlaying) SheetsSyncer.SyncEnemyData(); } }
```

E. Level Load

Additive Scenes

Why?	While the main gameplay scene (Core) stays loaded, level scenes (FireLand, IceWorld) are added on top. You don't need to duplicate persistent objects like HUD or GameManager in every scene.
Without it?	Every level change resets the entire scene: Singletons reinitialise, load times grow, and a flash screen appears.
Where?	Core (persistent systems) + Level_X (geometry + enemies) + UI (overlay) — a layered structure.

In-Game Usage Example

```
public class SceneLoader : MonoBehaviour { private string _currentLevel; public async Task
LoadLevel(string sceneName) { // Unload the old level first if (!string.IsNullOrEmpty(_currentLevel)) {
var unload = SceneManager.UnloadSceneAsync(_currentLevel); await unload; } // Load the new level
additively var load = SceneManager.LoadSceneAsync(sceneName, LoadSceneMode.Additive); await load;
_currentLevel = sceneName; SceneManager.SetActiveScene(SceneManager.GetSceneByName(sceneName)); } }
```

Async Loading & Loading Screen

Why?	Async loading doesn't block the main thread; a loading screen, progress bar, or animation keeps running smoothly. allowSceneActivation lets you delay the scene transition until loading completes.
Without it?	Synchronous LoadScene = freeze, black screen. The player wonders if it crashed. Long levels can cause 5–10 seconds of total freeze.
Where?	All level transitions and large asset bundle loads.

In-Game Usage Example

```
IEnumerator LoadWithProgress(string scene) { _loadingScreen.SetActive(true); var op =
SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive); op.allowSceneActivation = false; // Wait until
fully ready while (op.progress < 0.9f) { _progressBar.value = op.progress; yield return null; }
_progressBar.value = 1f; yield return new WaitForSeconds(0.3f); // Short wait for smooth UX
op.allowSceneActivation = true; _loadingScreen.SetActive(false); }
```

Memory Cleanup

Why?	After scene unload, chaining Addressables.Release + Resources.UnloadUnusedAssets + GC.Collect keeps memory clean and prevents leaks in long play sessions.
Without it?	Even after scenes are unloaded, referenced assets remain in memory. In long sessions or on mobile, a crash becomes inevitable.
Where?	Level transition, after boss fight, return to main menu.

In-Game Usage Example

```
async Task CleanupAfterLevelUnload() { // 1. Release Addressable assets foreach (var h in _levelHandles)
if (h.IsValid()) Addressables.Release(h); _levelHandles.Clear(); // 2. Unload unused assets var op =
```

```
Resources.UnloadUnusedAssets(); await op; // 3. Clean managed heap System.GC.Collect();
Debug.Log("[Memory] Cleanup complete."); }
```

F. Animation Curves

Easing

Why?	Linear movement looks robotic. Applying ease-in/ease-out via AnimationCurve makes UI animations, camera movement, and object interpolation feel professional.
Without it?	Everything moves at constant speed: opening panels feel mechanical, jumping enemies look like robots. The game feels 'lifeless'.
Where?	UI panel open/close, camera follow, boss entrance animation, skill indicator scaling.

In-Game Usage Example

```
public class UIPanel : MonoBehaviour { [SerializeField] private AnimationCurve _openCurve; // ease out
[SerializeField] private float _duration = 0.3f; public IEnumerator Open() { float t = 0; Vector3 target =
Vector3.one; transform.localScale = Vector3.zero; while (t < _duration) { t += Time.deltaTime; float
normalized = t / _duration; float curved = _openCurve.Evaluate(normalized); transform.localScale =
Vector3.LerpUnclamped(Vector3.zero, target, curved); yield return null; } transform.localScale = target; }
}
```

Stat Progression Curves

Why?	Power growth from level 1 to 100 shouldn't be linear. With curves you can design fast early-level progression that slows at higher levels, or exponential growth.
Without it?	You're locked into a linear formula. No balancing flexibility; every level feels the same.
Where?	HP, damage, speed stat growth per level; XP gain curve; price scaling.

In-Game Usage Example

```
[CreateAssetMenu(menuName="Game/StatCurve")] public class StatCurveData : ScriptableObject { public
AnimationCurve hpCurve; // X: level (0-1), Y: multiplier public AnimationCurve damageCurve; public float
baseHP = 100f; public float baseDamage = 10f; public float GetHP(int level, int maxLevel) { float t =
(float)level / maxLevel; return baseHP * hpCurve.Evaluate(t); } } // Lv1: 100 HP | Lv50: 800 HP | Lv100:
5000 HP (exponential curve)
```

Movement Curves

Why?	Defining a projectile arc, enemy charge movement, or camera shake intensity with AnimationCurve makes motion design adjustable independently of code.
Without it?	Hardcoded velocity values. Changing a projectile arc requires calling a programmer.
Where?	Projectile parabola, ground slam movement, camera shake, platform jumping.

In-Game Usage Example

```
public class GroundSlam : MonoBehaviour { [SerializeField] private AnimationCurve _heightCurve; // jump
arc [SerializeField] private AnimationCurve _speedCurve; // speed profile [SerializeField] private float
```

```
_duration = 0.5f; IEnumerator SlamTo(Vector3 target) { float t = 0; Vector3 start = transform.position; while (t < _duration) { t += Time.deltaTime; float n = t / _duration; float height = _heightCurve.Evaluate(n); float speed = _speedCurve.Evaluate(n); Vector3 pos = Vector3.Lerp(start, target, speed); pos.y += height; transform.position = pos; yield return null; } } }
```

Spawn Pacing

Why?	Controlling enemy spawn rate with AnimationCurve creates dramatic rhythm: slow at wave start, dense in the middle, boss at the end.
Without it?	Constant spawn intervals = monotone gameplay. Players predict waves; tension disappears.
Where?	WaveManager, ambient enemies throughout a level, boss phase transitions.

In-Game Usage Example

```
public class WaveManager : MonoBehaviour { [SerializeField] private WaveData[] _waves; private int _waveIndex; private float _elapsed; private float _spawnBudget; void Update() { if (_waveIndex >= _waves.Length) return; var wave = _waves[_waveIndex]; _elapsed += Time.deltaTime; // Curve: X=0..1 (wave progress), Y=spawns/sec float t = _elapsed / wave.waveDuration; float rate = wave.spawnRateCurve.Evaluate(t); _spawnBudget += rate * Time.deltaTime; while (_spawnBudget >= 1f) { SpawnRandomEnemy(wave); _spawnBudget -= 1f; } if (_elapsed >= wave.waveDuration) NextWave(); } }
```

G. Save System

Binary Format Save

Why?	JSON is transparent and hackable; PlayerPrefs is too limited. BinaryWriter produces compact, unreadable save files — cheating is harder and I/O is faster.
Without it?	JSON save: players edit gold/health with a hex editor. PlayerPrefs can't store complex data and is unsuitable for large save files.
Where?	Player progress, inventory, level completion, settings, achievement data.

In-Game Usage Example

```
[System.Serializable] public class SaveData { public int version = 1; public float playerHP; public int gold; public List unlockedAbilities; public long saveTimestamp; } public static class SaveManager { static string Path => Application.persistentDataPath + "/save.dat"; public static void Save(SaveData data) { data.saveTimestamp = DateTimeOffset.UtcNow.ToUnixTimeSeconds(); using var fs = File.Open(Path, FileMode.Create); using var bw = new BinaryWriter(fs); var bytes = Encoding.UTF8.GetBytes(JsonUtility.ToJson(data)); bw.Write(bytes.Length); bw.Write(bytes); } public static SaveData Load() { if (!File.Exists(Path)) return new SaveData(); using var br = new BinaryReader(File.OpenRead(Path)); return JsonUtility.FromJson( Encoding.UTF8.GetString(br.ReadBytes(br.ReadInt32()))); } }
```

Versioning & Migration

Why?	Save format may change with game updates. A version number lets you migrate old saves to the new format so player progress is never lost.

Without it?	When the save format changes, old saves appear corrupt. Players lose progress = negative store reviews.
Where?	The save system should include a version bump + migration function for every major data change.

In-Game Usage Example

```
public static SaveData Migrate(SaveData old) { switch (old.version) { case 0: // V0 -> V1: 'unlockedAbilities' field added old.unlockedAbilities = new List { "dash" }; old.version = 1; goto case 1; case 1: // V1 -> V2: 'currencies' dict replaces 'gold' // old.currencies = new Dict { { "gold", old.gold } }; old.version = 2; goto case 2; case 2: // current version break; } return old; } // Migrate immediately on load: // var data = SaveManager.Load(); // data = Migrate(data);
```

Meta Data

Why?	Save file metadata (save date, play time, level, screenshot) gives players information when choosing a slot and provides context needed for cloud sync.
Without it?	Players can't tell which slot to pick and may overwrite the wrong one. Cloud sync can't determine which save is newer.
Where?	Save slot selection screen, cloud sync conflict resolution, analytics.

In-Game Usage Example

```
[System.Serializable] public class SaveMeta { public int version; public long timestamp; // Unix epoch public float totalPlaySeconds; public int currentLevel; public string heroName; public string screenshotBase64; // slot screenshot } void DisplaySlot(SaveMeta meta) { var date = DateTimeOffset.FromUnixTimeSeconds(meta.timestamp); _slotText.text = $"{meta.heroName} - Lv {meta.currentLevel}"; _dateText.text = date.ToString("dd MMM yyyy HH:mm"); _timeText.text = TimeSpan.FromSeconds(meta.totalPlaySeconds) .ToString(@"hh\:mm"); }
```

H. Input System

New Unity Input System

Why?	The old Input.GetKey() approach is polling-based and multi-device support is painful. The new system is event-driven: InputAction makes gamepad, keyboard, and touch work with the same code.
Without it?	Adding gamepad support means rewriting from scratch. Input rebinding is extremely complex with the old system.
Where?	Player controller, UI navigation, debug cheats, replay system.

In-Game Usage Example

```
public class PlayerInput : MonoBehaviour { private GameControls _controls; void Awake() { _controls = new GameControls(); _controls.Player.Jump.performed += OnJump; _controls.Player.Attack.performed += OnAttack; _controls.Player.Move.performed += OnMove; } void OnEnable() => _controls.Enable(); void OnDisable() => _controls.Disable(); void OnJump(InputAction.CallbackContext ctx) { _playerController.Jump(); } void OnMove(InputAction.CallbackContext ctx) { Vector2 dir = ctx.ReadValue(); _playerController.SetMoveDirection(dir); } }
```

Gamepad & Keyboard Support

Why?	Binding the same Action to both WASD and the gamepad left stick in InputActionAsset supports both without platform differences. Control schemes let you detect which device is active.
Without it?	A game written only for keyboard: plug in a gamepad and nothing works. You have to rewrite input from scratch for console/mobile ports.
Where?	Every project targeting PC + console release. Mobile touch can be bound to the same Action.

In-Game Usage Example

```
public class PlayerController : MonoBehaviour { private Vector2 _moveInput; // Input System callbacks public void OnMove(InputValue value) => _moveInput = value.Get(); public void OnJump(InputValue value) { if (value.isPressed) DoJump(); } // Detect active device public void OnControlsChanged(UnityEngine.InputSystem.PlayerInput pi) { bool isGamepad = pi.currentControlScheme == "Gamepad"; _hud.ShowGamepadIcons(isGamepad); } }
```

I. Quick Test

Start from Any Environment / Stage

Why?	Testing a bug in level 10 while having to play through 1–9 is a waste of time. A debug launcher lets you spawn directly at 500m in Fire Land.
Without it?	Every test cycle requires playing from the beginning. Testing late-game content becomes nearly impossible.
Where?	Editor-only DebugLauncher script. Don't include in builds — use a platform directive.

In-Game Usage Example

```
#if UNITY_EDITOR [CreateAssetMenu(menuName="Debug/LaunchConfig")] public class DebugLaunchConfig : ScriptableObject { public string targetScene = "FireLand"; public float startDistance = 500f; public string heroId = "warrior"; public int startGold = 9999; public bool enableDebugHUD = true; } [InitializeOnLoad] public static class QuickLauncher { static QuickLauncher() { EditorApplication.playModeStateChanged += OnPlay; } static void OnPlay(PlayModeStateChange state) { if (state != PlayModeStateChange.EnteredPlayMode) return; var cfg = Resources.Load("DebugLaunchConfig"); if (cfg != null) GameBootstrapper.StartWithConfig(cfg); } } #endif
```

Inventory Override & Time Control

Why?	Being able to start the game in a specific state — a particular item combination to test a boss, or night mode — is critical.
Without it?	Collecting every combination in-game takes hours. Testing time-based mechanics becomes nearly impossible.
Where?	Debug launcher + CheatConsole (active only in editor/dev builds).

In-Game Usage Example

```
public class CheatConsole : MonoBehaviour { void Update() { #if DEVELOPMENT_BUILD || UNITY_EDITOR if (Input.GetKeyDown(KeyCode.F1)) GiveAllItems(); if (Input.GetKeyDown(KeyCode.F2)) SetGameTime(22, 0); //
```

```
night if (Input.GetKeyDown(KeyCode.F3)) Time.timeScale = 5f; // x5 speed if (Input.GetKeyDown(KeyCode.F4)) KillAllEnemies(); #endif } void SetGameTime(int hour, int minute) { GameTimeManager.Instance.SetTime(hour, minute); Debug.Log($"[Cheat] Game time set to {hour:00}:{minute:00}"); } }
```

Stress Testing

Why?	Placing 100+ enemies and 1000 projectiles in a scene lets you identify bottlenecks early. The Unity Profiler reveals which system is the choke point.
Without it?	Performance issues are only discovered through player complaints. Late-stage optimisation is far more costly.
Where?	Editor-only stress test scene; automated performance baseline tests in a CI pipeline.

In-Game Usage Example

```
#if UNITY_EDITOR public class StressTest : MonoBehaviour { [SerializeField] private int _enemyCount = 200; [SerializeField] private int _projectileRate = 50; // per second [ContextMenu("Run Stress Test")] void Run() { StartCoroutine(SpawnEnemies()); StartCoroutine(FireProjectiles()); } IEnumerator SpawnEnemies() { for (int i = 0; i < _enemyCount; i++) { EnemyFactory.Instance.Create("goblin", Random.insideUnitSphere * 20f); if (i % 10 == 0) yield return null; // spread across frames } } } #endif
```

J. Localization

Unity Localization System

Why?	Unity's official Localization package manages text and visuals independently of language via String/Asset tables. It supports runtime language switching, Google Sheets sync, and plural rules.
Without it?	You start with a homemade string dict; it becomes unmanageable at scale. Features like plurals (1 apple / 3 apples), RTL language support, and font overrides are absent.
Where?	All UI text, item names, dialogues, achievement descriptions.

In-Game Usage Example

```
public class LocalizedLabel : MonoBehaviour { [SerializeField] private LocalizedString _key; [SerializeField] private TMP_Text _text; void OnEnable() => _key.StringChanged += UpdateText; void OnDisable() => _key.StringChanged -= UpdateText; void UpdateText(string value) => _text.text = value; } // Change language at runtime: // using UnityEngine.Localization.Settings; // LocalizationSettings.SelectedLocale = // LocalizationSettings.AvailableLocales.GetLocale("tr");
```

Key Naming: domain.object.field

Why?	Without a naming standard, searching thousands of localisation keys is impossible and duplicates multiply. The domain.object.field format enables hierarchical browsing.
Without it?	Meaningless keys like btn1, text_23, msg_attack. Context disappears as the language file grows and wrong strings get used.
Where?	All string table keys must follow this format. Automation scripts should enforce it too.

In-Game Usage Example

```
// Format: domain.object.field // domain -> ui, gameplay, item, enemy, hud, menu, dialog // object -> element on screen // field -> label, desc, tooltip, title, button, error // ui.mainmenu.start_button -> 'Start Game' // ui.mainmenu.settings_button -> 'Settings' // ui.hud.health_label -> 'Health' // ui.shop.buy_button -> 'Buy' // gameplay.ability.dash.name -> 'Quick Run' // gameplay.ability.dash.desc -> 'Runs {0} units in {1} seconds' // item.sword_iron.name -> 'Iron Sword' // item.sword_iron.desc -> 'Simple but reliable.' // enemy.goblin.name -> 'Goblin' // dialog.npc.blacksmith.intro -> 'Hello, looking for a weapon?' // error.save.failed -> 'Save failed.'
```